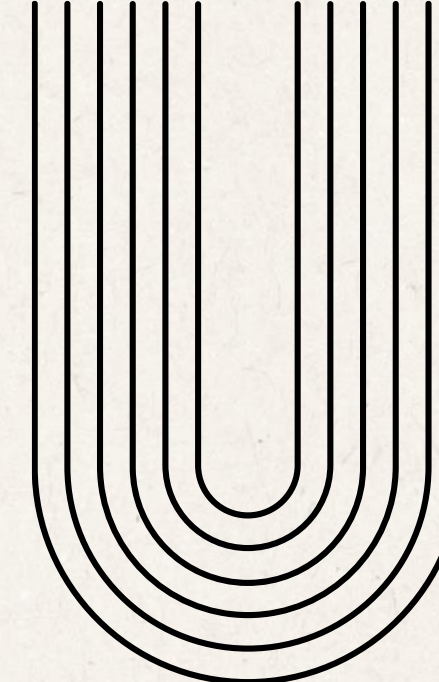


11/02/2026



BUBBLE SORT PARALLELO

Progetto Sistemi di Elaborazione Accelerata M

PRESENTED BY:

Gregorio Bussolari
Andrea Pivetti

Problema affrontato: Ordinamento array tramite algoritmo “bubble sort”

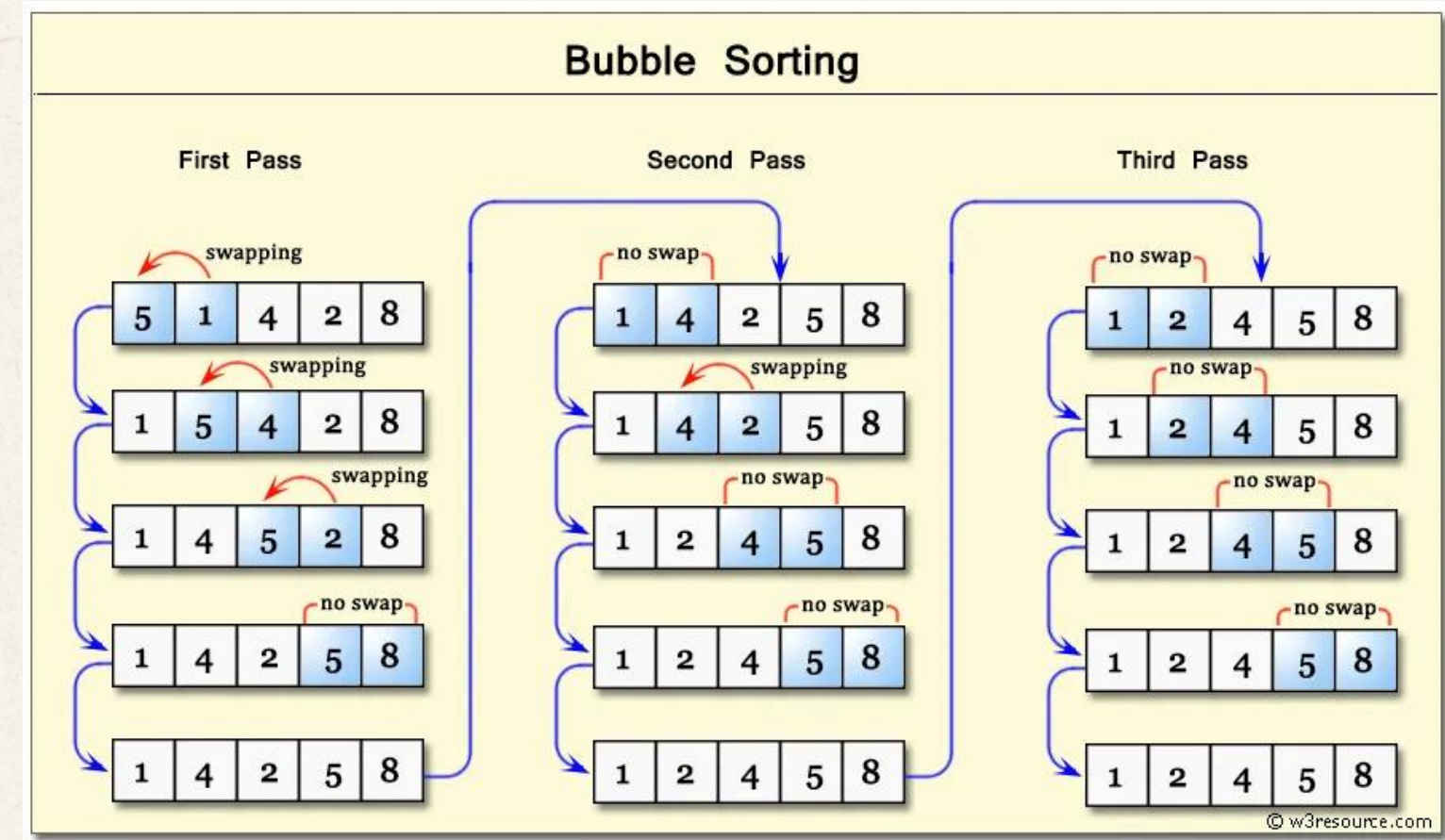
Algoritmo di ordinamento con complessità $O(N^2)$.

Gli elementi dell'array devono appartenere tutti allo stesso insieme, dotato necessariamente di una relazione d'ordine totale.

Nell'implementazione, ogni iterazione dipende dalla precedente (forte dipendenza di dato)

Questo rende più delicato il processo di parallelizzazione con i paradigmi SIMT e SIMD, dove potrebbero sorgere problemi di sincronizzazione e/o divergenza.

```
//pseudocode
procedure BubbleSort(A)
  scambio = true
  while scambio do
    scambio = false
    for i = 0 to length(A)-1 do
      if A[i] > A[i+1] then
        swap( A[i], A[i+1] )
        scambio = true
```

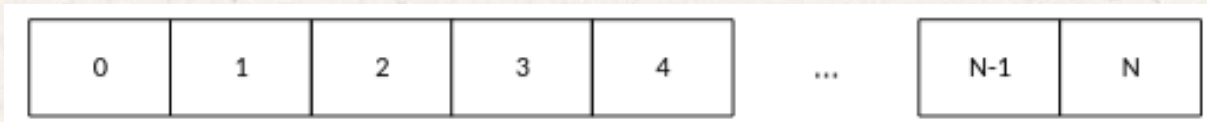


Decomposizione del problema in CUDA

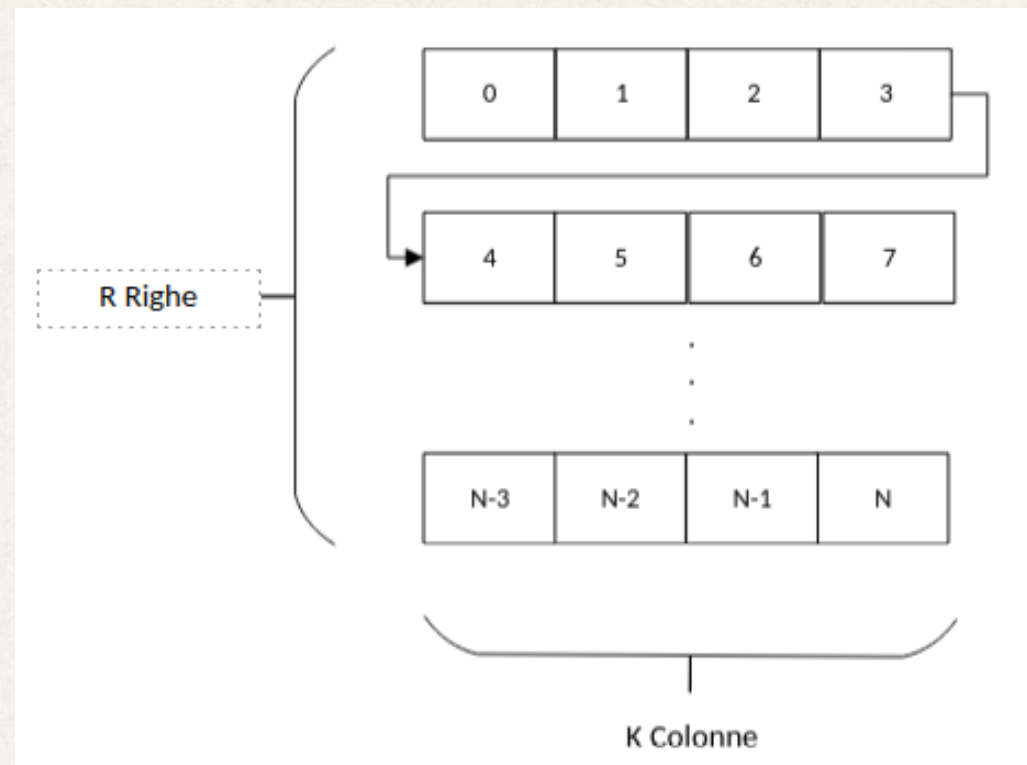
Strategia di decomposizione dell' algoritmo Bubble Sort:

Livello di Dominio

Dato un Array di N elementi



Questo può essere organizzato logicamente in “R” righe di “K” colonne



Livello di Logico

Ognuna delle righe può essere assegnata a un diverso thread, che ha il solo compito di eseguire K-1 confronti, ordinando un elemento della riga, operazione che può essere compiuta in parallelo da N thread



Problema:

Necessità di “rompere” il confine tra le righe, portando elementi più grandi nelle ultime righe della matrice e quindi dell’array



Soluzione:

Al termine di ogni fase di ordinamento righe, deve corrispondere una fase di “check” dei confini tra le righe, in cui N-1 thread confrontano l’ultimo elemento della riga con il primo della riga successiva



Conseguenza:

Sequenzializzazione delle fasi di ordinamento delle righe e di controllo dei confini tra righe diverse

Gerarchia dei Thread e accesso ai dati

L'organizzazione gerarchica seguita prevede un assunto di base:
a un thread è assegnata una riga della matrice logica risultante.

N: il numero di elementi dell'array

K: il numero di elementi in ogni riga (colonne)

R: numero di righe con cui l'array è logicamente diviso

$$R = \text{ceil}(N / K)$$

Abbiamo scelto creare blocchi 2D di
dimensione parametrizzabile
(64, 256, 512):

dim3 block(blockSizeX, blockSizeY);

Pertanto il numero di blocchi risulta

$$\text{Numero di blocchi} = \text{ceil}(R / \text{blockSize})$$

Abbiamo calcolato una griglia 2D, ottenuta nel seguente modo

$$\text{gridDimX} = \text{ceil}(\text{sqrt}(\text{Numero di blocchi}))$$

$$\text{dim3 grid}(\text{gridDimX}, (\text{num_blocchi} + \text{gridDimX} - 1) / \text{gridDimX})$$

Accesso ai dati

Nell'ordinamento di una riga ogni thread deve
calcolare il primo elemento della riga su cui opera

$$\text{Start Index} = \text{Global Index} * K$$

Nel controllo dei confini, ogni thread accede
all'ultimo elemento della sua riga e al primo della
riga successiva

Prima Implementazione Parallela Naïve

Spiegazione dell'algoritmo

Sono stati realizzati due kernel per implementare l'ordinamento:

1. **Ordina_riga:** kernel in cui un thread ha il solo compito di svolgere $K-1$ confronti con al più $K-1$ scambi, per portare l'elemento più grande nell'ultima posizione della riga. Complessità $O(K)$
1. **Check_confini:** kernel in cui un thread deve confrontare l'ultimo elemento della riga con il primo della riga successiva. Complessità $O(1)$

Queste due operazioni devono essere necessariamente serializzate, pertanto ogni volta che viene terminato il primo Kernel, si torna alla CPU, si attende la terminazione di tutti i thread, e si esegue anche il secondo. Complessità risultante: $O(K) + O(1) = O(K)$

Queste due operazioni devono essere svolte un totale di N volte per ottenere l'array completamente ordinato. Pertanto la complessità totale risulta $O(N * K)$

Problemi ed inefficienze

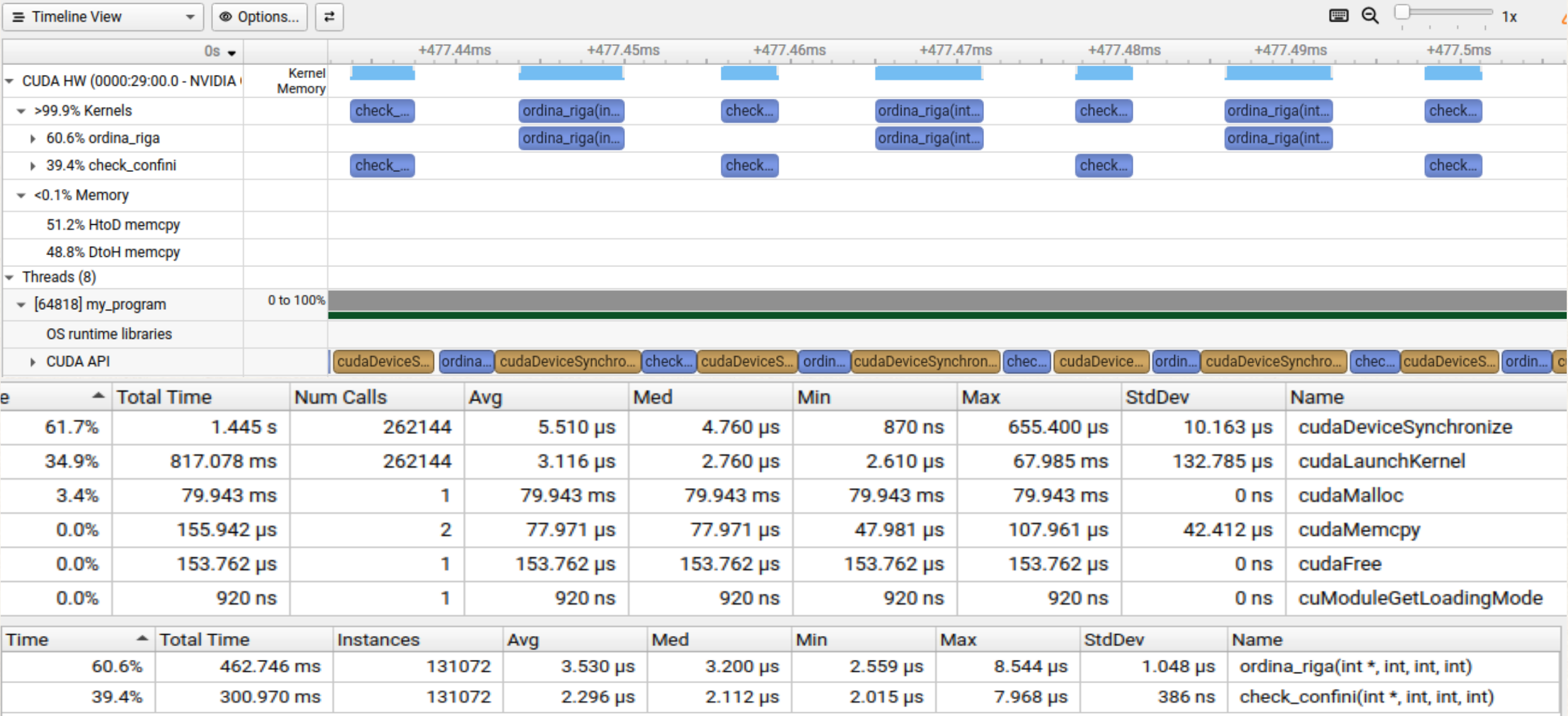
- Il passaggio continuo da CPU a GPU, nel lancio di kernel successivi peggiora notevolmente le prestazioni
- La complessità dipende dal numero di colonne in cui viene diviso l'array di elementi
- Il lancio di N Kernel distinti rendono poco pratica l'analisi con Nsight Compute
- Il lancio di N kernel è molto costoso in termini di latenza

Prima Implementazione Parallela Naïve: Nsight System

Profiling a livello di sistema:

Il test è stato eseguito su un array di 131k elementi interi (500 KB di dati)

Nota: Device di riferimento GTX 1660 S
Compute Capabilty 7.5
all’interno dell’architettura Turing



Notiamo che, come avevamo previsto, questa configurazione non è ottimale. Sia il lancio dei kernel sia la *cudaDeviceSynchronize()* rappresentano un grande collo di bottiglia.

Il tempo totale nel quale i due kernel sono in esecuzione sulla GPU (**763ms**) è minore del tempo che la CPU ci mette a lanciare i kernel stessi (**817ms**).

Il tempo di cudaDeviceSincronize() comprende l'esecuzione dei kernel sulla GPU. Il **52,8%** del tempo passato in questa API è dovuto a overhead di sistema.

Dim. Array	Tgpu(ms)	Tcpu(ms)	Speedup	Dim. Blocco
8k (32KB)	112.551	203.187	1.805	256
16K (64KB)	862.332	12343.165	14.314	256
131K(500KB)	1899.040	49045.695	25.827	256

Ottimizzazione Incrementale I

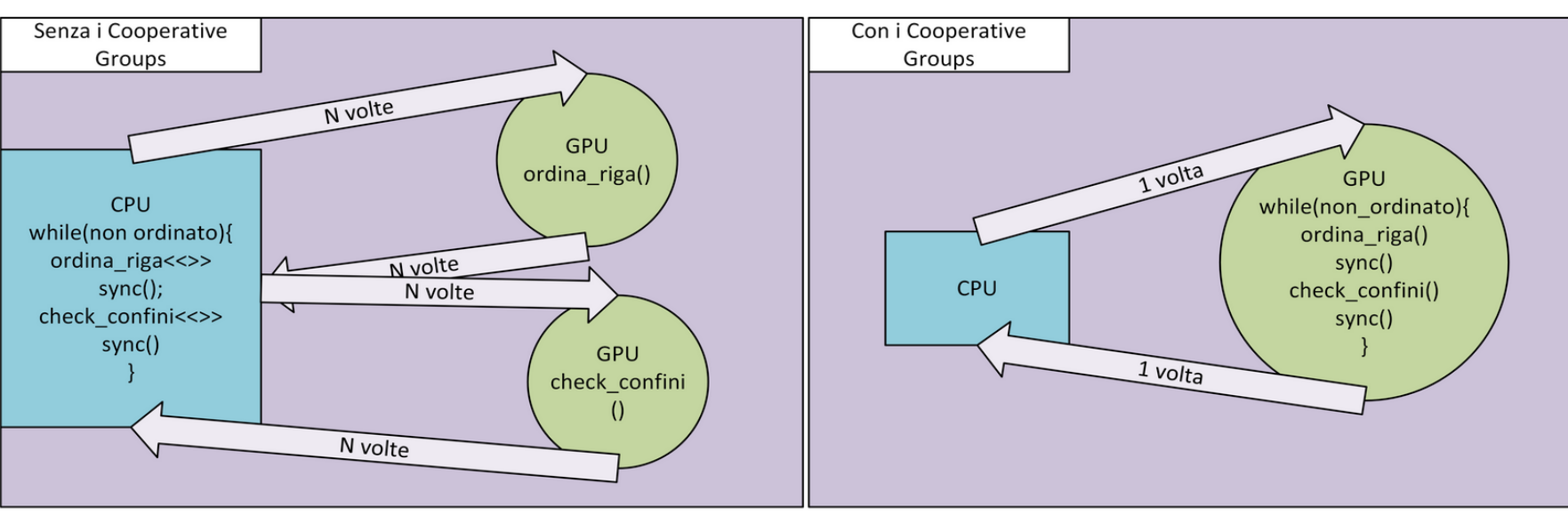
Cooperative Groups

Questa seconda versione mira a ridurre l’overhead dovuto ai frequenti passaggi tra CPU e GPU, causati dagli API di lancio e di sincronizzazione tra kernels.

Questo risultato è stato ottenuto sfruttando l’API Cooperative Groups di CUDA. Questo API permette di definire gruppi di thread sui quali si può effettuare la sincronizzazione. Questo API permette anche di **sincronizzare tutti i thread di una griglia**, funzionalità perfetta per il caso in esame.

Vantaggi

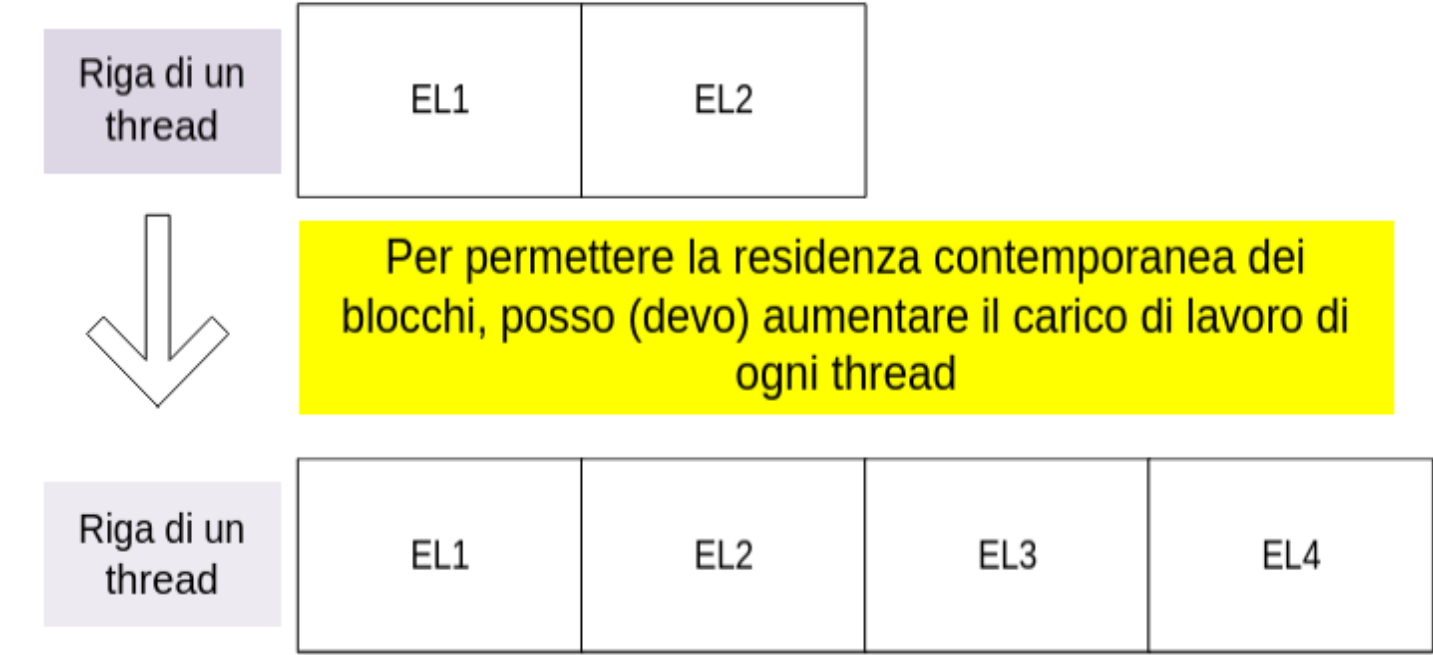
Grazie a questo API è possibile ovviare il collo di bottiglia introdotto precedentemente, sincronizzando i thread della griglia sulla GPU, senza dover tornare ogni volta sulla CPU.



Limitazioni

Una limitazione dei CG è che necessariamente tutti i blocchi lanciati devono poter essere **contemporaneamente residenti sugli SM della GPU**.

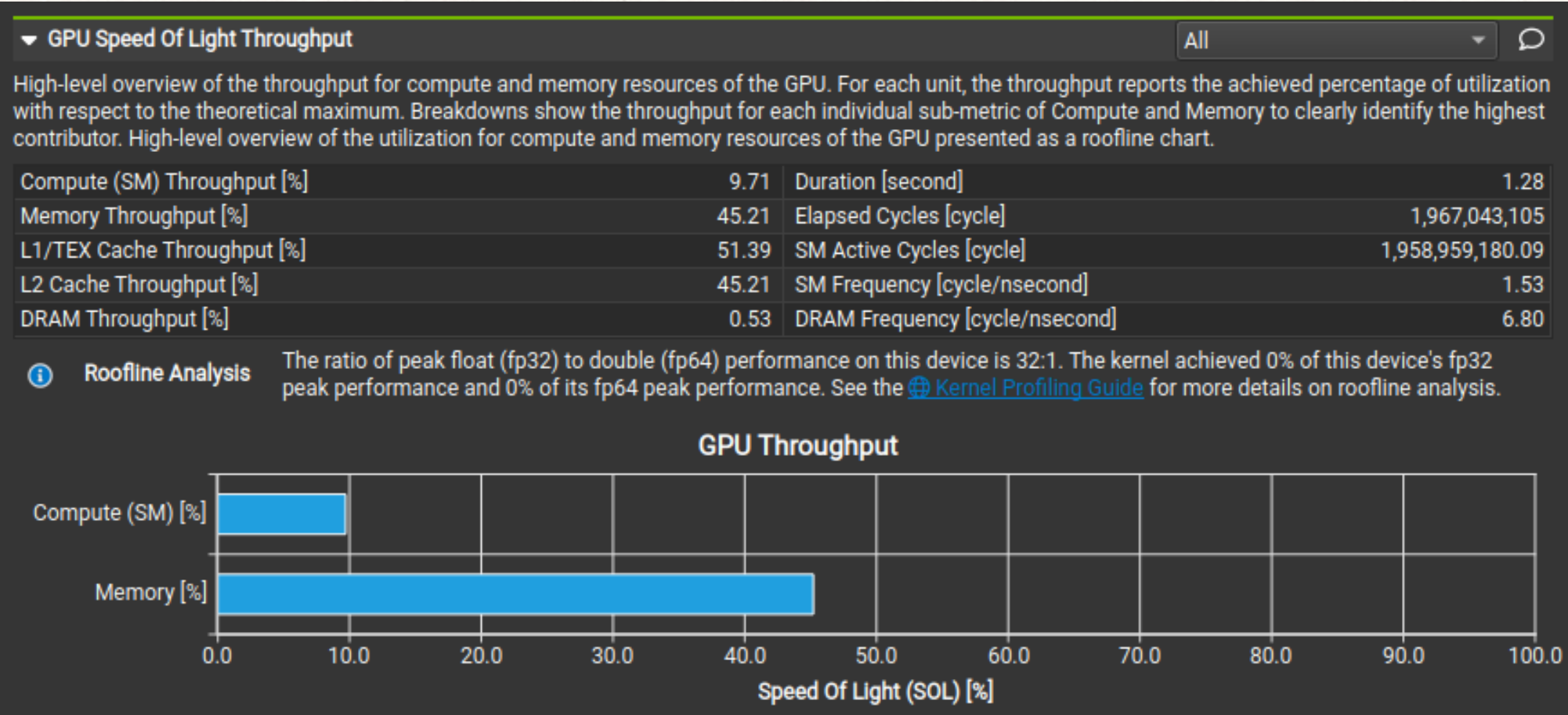
Nel caso in cui il numero di blocchi fosse troppo grande per soddisfare questo parametro, sarebbe necessario modulare il carico di lavoro per thread. Aumentando le colonne diminuisce però il parallelismo e aumenta la complessità computazionale [O(K*N)].



Ottimizzazione Incrementale I

Analisi dei risultati: Nsight Compute

Il kernel di ordinamento non esegue operazioni in virgola mobile e presenta un’arithmetic intensity trascurabile. L'analisi Roofline non è significativa per la valutazione delle prestazioni. Il kernel è certamente **Memory Bound**, le implementazioni successive mirano a migliorare l'accesso in memoria.



Grid Size	64
Registers Per Thread [register/thread]	30
Block Size	256
Threads [thread]	16,384
Waves Per SM	0.73

Il blocco considerato è di 256 thread. Il numero di Waves per SM è minore di 1 a causa del numero di blocchi che sono stati lanciati, il quale non è sufficiente per occupare nemmeno una volta gli SM della GPU.

Theoretical Occupancy [%]	100	Block Limit Registers [block]	8
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	16
Achieved Occupancy [%]	72.73	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	23.27	Block Limit SM [block]	16

L'occupancy ottenuta è circa pari al 73%, discostandosi dal valore teorico, per via del carico di lavoro sbilanciato tra i blocchi e dal numero insufficiente di blocchi lanciati (limitazione dei Cooperative Groups). Poiché 4 blocchi da 8 warp (256 thread) occupano già tutti i 32 slot dei warp, il limite dei registri non viene mai raggiunto. Questo implica che un utilizzo minore dei registri non porterebbe aumenti dell'occupancy ottenuta.

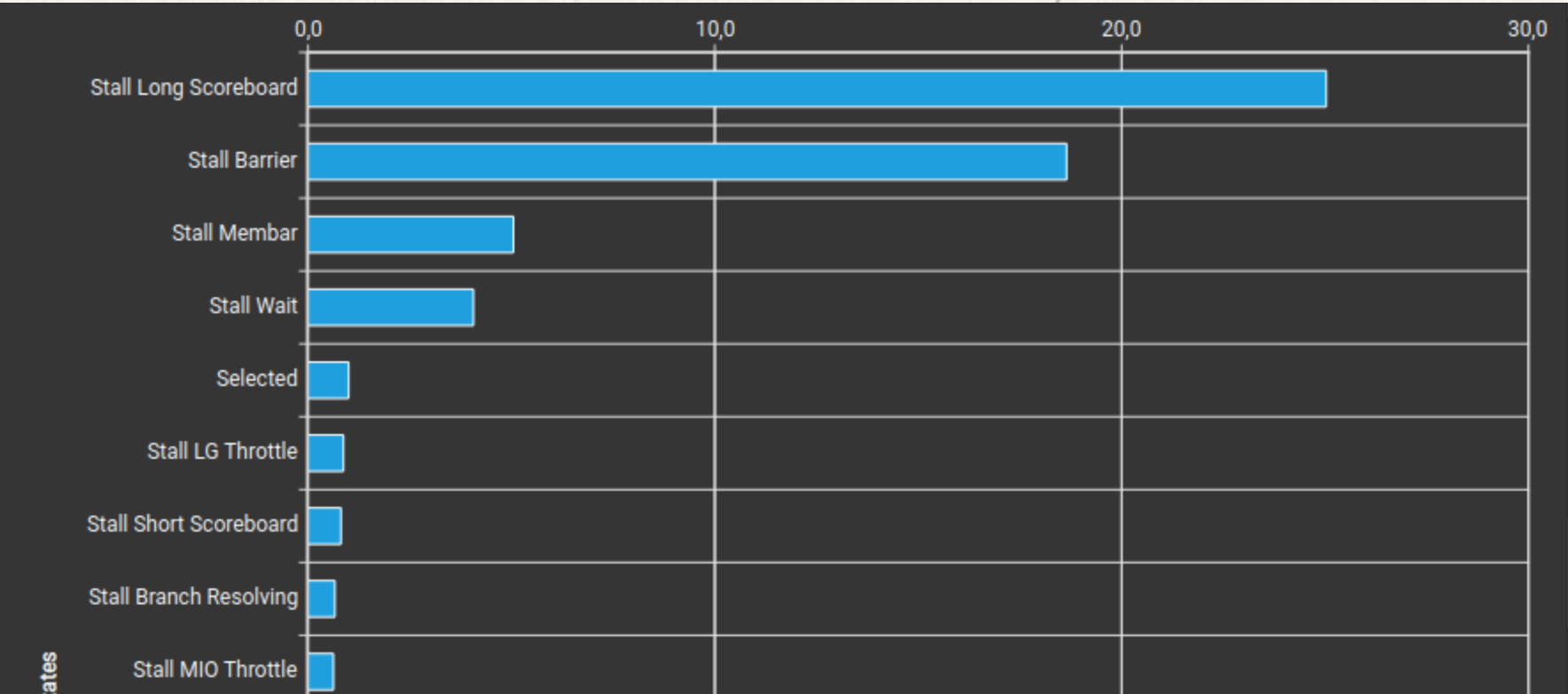
Parametro	Valore
Compute Capability	7.5
Max Threads per Block	1024
Max Block Dimensions	1024, 1024, 64
Max Grid Dimensions	2 ³¹ -1, 65535, 65535
Warp Size	32
Max Registers per Block	65536
Max Registers per Thread	255
Shared Memory per Block	64 KB
Shared Memory per SM	96 KB
Max Blocks per SM	16
Max Threads per SM	1024

Ottimizzazione Incrementale I

Analisi dei risultati: Nsight Compute

► Source Counters				All	
Source metrics, including branch efficiency and sampled warp stall reasons. Warp Stall Sampling metrics are periodically sampled over the kernel runtime. They indicate when warps were stalled and couldn't be scheduled. See the documentation for a description of all stall reasons. Only focus on stalls if the schedulers fail to issue every cycle.					
Branch Instructions [inst]	579,055,146	Branch Efficiency [%]	97.89		
Branch Instructions Ratio [%]	0.15	Avg. Divergent Branches	96,069.82		

Executed Ipc Elapsed [inst/cycle]	0,40
Executed Ipc Active [inst/cycle]	0,40
Issued Ipc Active [inst/cycle]	0,40



Il kernel è dominato dall' esecuzione di load/store in memoria e dai confronti tra gli elementi dell'array. Si può notare una bassa divergenza intra-warp: codice esegue moltissimi branch ma la maggior parte dei warp segue lo stesso percorso di esecuzione

Le principali limitazioni di questa implementazione derivano dal fatto che, usando Cooperative Groups, tutti i blocchi devono essere residenti contemporaneamente sugli SM. Questo vincolo riduce l'occupazione effettiva, pur annullando completamente il tail effect, poichè il numero di blocchi lanciati potrebbe risultare inferiore al numero di SM.

Il kernel inoltre fortemente synchronization bound: a ogni iterazione dell'algorithmo vengono eseguite due grid.sync(), che introducono cicli di stallo molto elevati e determinano un IPC (instruction per cycle) estremamente basso (0.4).

Lo Stall Chart mostra che le principali cause di stall sono Long Scoreboard e Barrier e Stall Membar. Gli stalli causati da Long Scoreboard rappresentano cicli di attesa dovuti ad accessi in memoria ad alta latenza, mentre Barrier e Membar sono attese attribuibili alle chiamate di sincronizzazione. Nonostante la sincronizzazione sia un requisito necessario intrinseco della natura del problema, gli stalli dovuti ad accessi in memoria ad alta latenza possono essere diminuiti tramite l'uso della shared memory, che sarà introdotta nelle prossime slide.

Ottimizzazione Incrementale II

Shared Memory

Come prima cosa, al fine di ottimizzare il pattern di accesso alla memoria globale della soluzione precedente, è stata realizzata una versione che utilizza la shared memory

Obiettivo: usare il minor numero di transazioni in memoria globale per servire il massimo numero di richieste in memoria.

Idea Di Base:

La shared memory viene utilizzata per allocare dinamicamente una riga della matrice, così da eseguire tutti i confronti in SMEM durante il kernel ed effettuare una sola scrittura finale in memoria globale.



Problema

La shared memory viene allocata per blocco, senza possibilità di sincronizzazione tra SMEM di blocchi diversi, complicando di fatto l'ordinamento dell'array finale



Soluzione

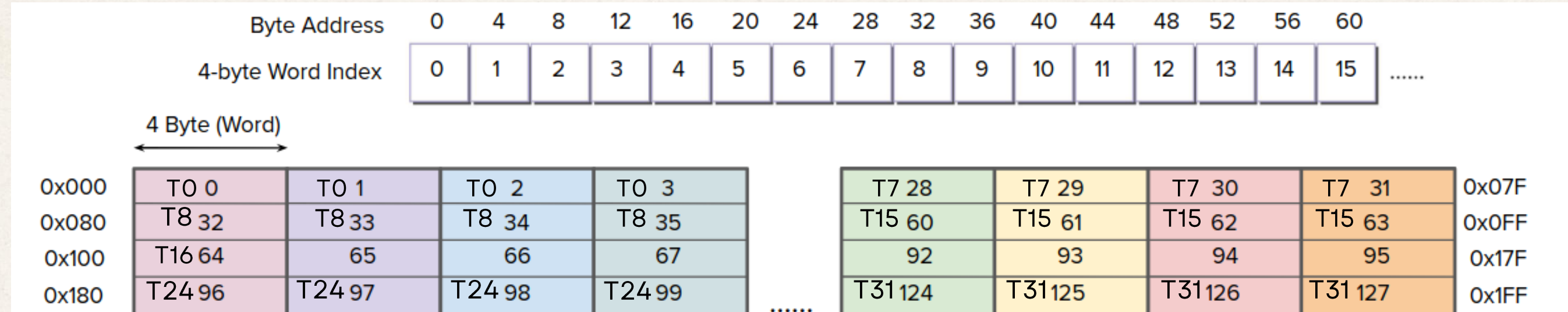
La sincronizzazione tra blocchi avviene attraverso la global memory e viene fatta soltanto in fase di controllo dei confini tra due righe, dai thread che si trovano al confine tra due blocchi. Negli altri casi non è necessaria, poiché le righe appartenenti allo stesso blocco sono indipendenti.

DIM SMEM = block.x * block.y * K * sizeof(int)

Shared Memory

Gestione della Shared Memory: caso in esame, si considera l'utilizzo di righe formate da 4 colonne

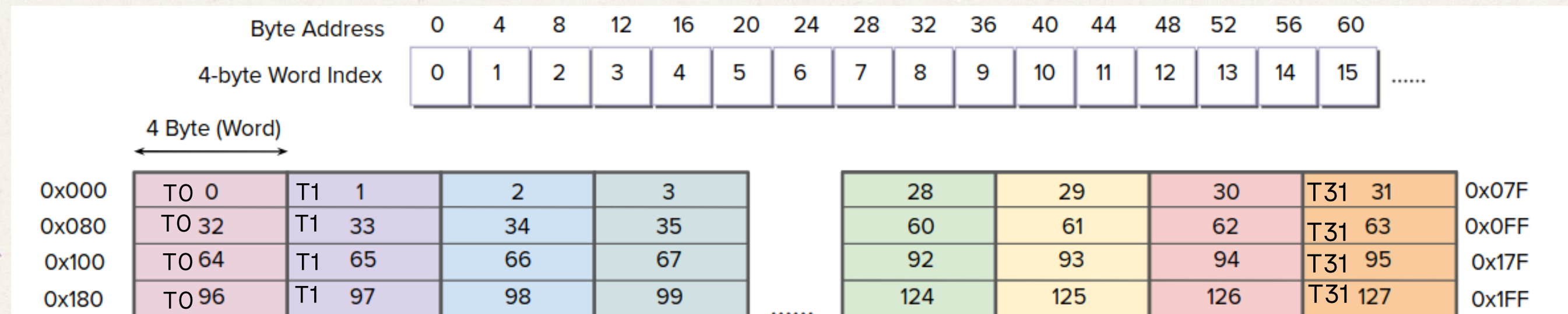
Thread	1	2	0	4
T0	E1	E2	E0	E3
T1	E5	E6	E4	E7



4-way Bank Conflict!

Gestione della Shared Memory: Caso Ideale

Thread	1	2	0	4
T0	E1	E2	E0	E3
T1	E5	E6	E4	E7



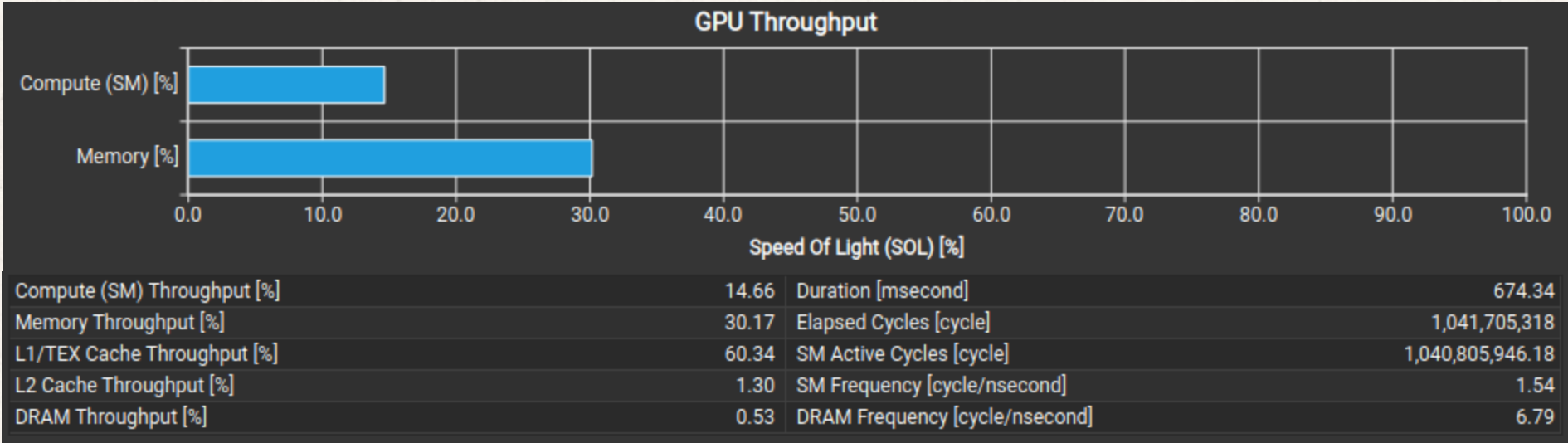
Abbiamo deciso di adottare comunque il caso con 4-way Conflict, sacrificando in parte l'efficienza in favore di chiarezza e semplicità

Ottimizzazione Incrementale II

Analisi dei risultati: Nsight Compute

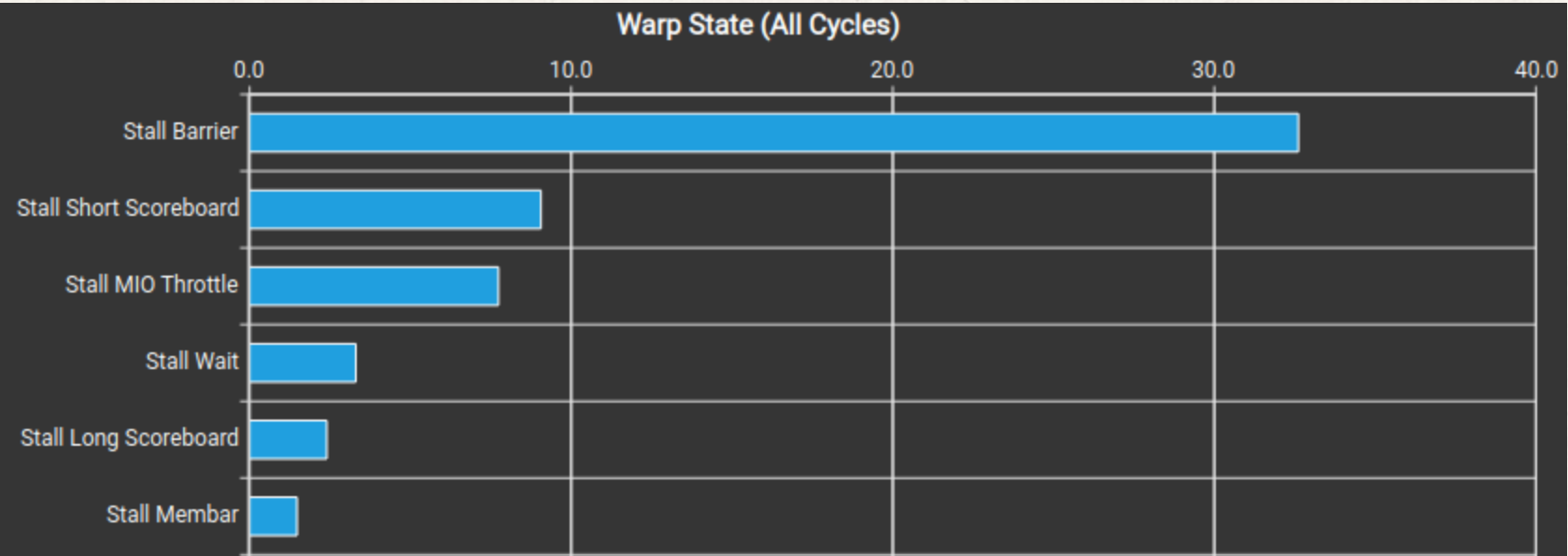
Theoretical Occupancy [%]	100	Block Limit Registers [block]	4
Theoretical Active Warps per SM [warp]	32	Block Limit Shared Mem [block]	8
Achieved Occupancy [%]	72.73	Block Limit Warps [block]	4
Achieved Active Warps Per SM [warp]	23.27	Block Limit SM [block]	16

Vediamo che, relativamente all'occupancy, non abbiamo cambiamenti in quanto. Il limite imposto dai registri per lo scheduling dei blocchi è diminuito a causa delle nuove variabili di gestione, mentre la shared memory è diventata un vincolo hardware attivo.



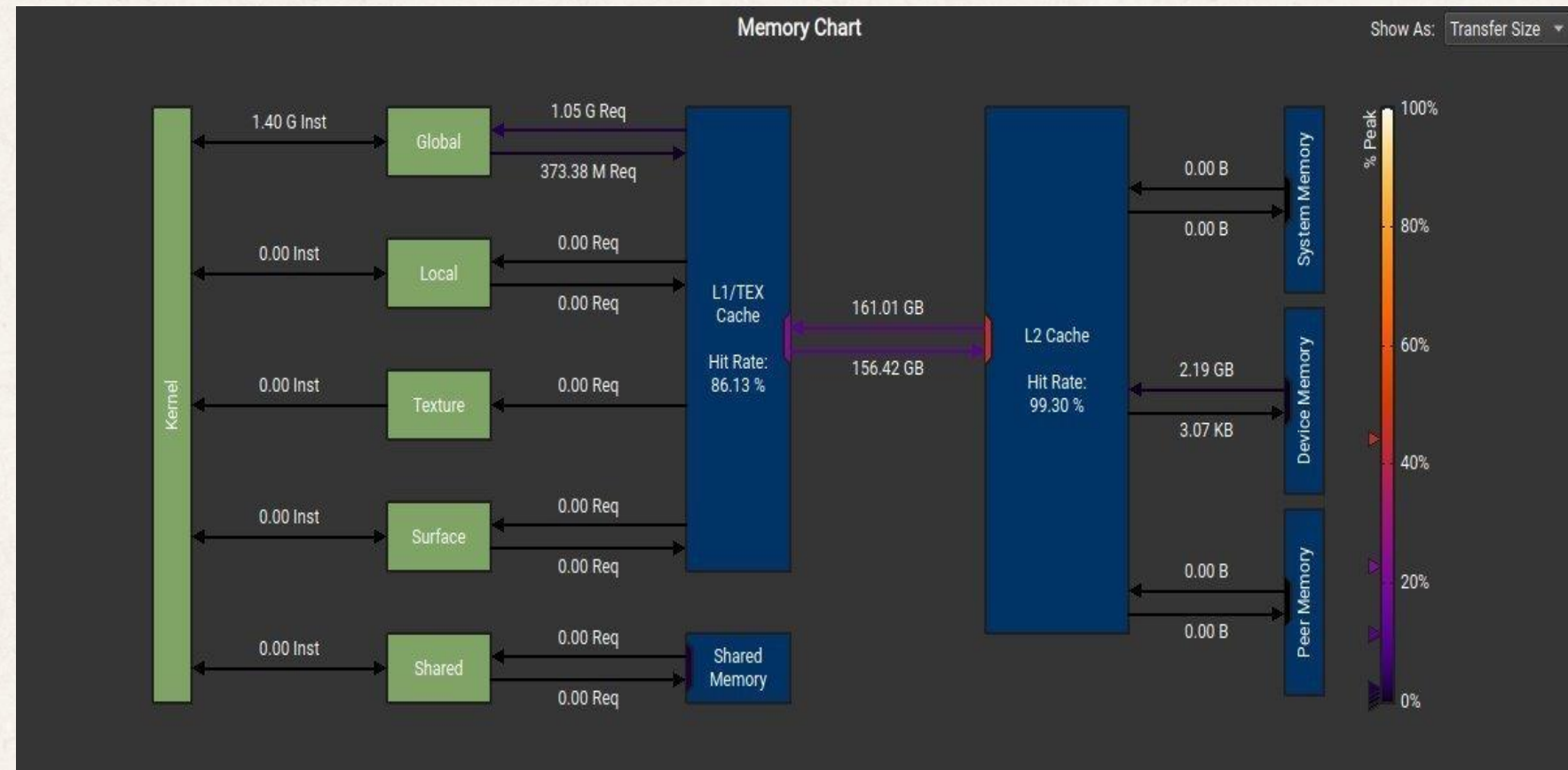
Il memory throughput invece diminuisce, andando da 45% a 30%. Questo è dovuto al fatto che la maggior parte del traffico si sposta sulla shared memory. Allo stesso tempo il compute throughput è leggermente aumentato, in quanto presumibilmente diminuiscono gli stalli dovuti agli accessi a memoria lenta da parte degli SM.

Come possiamo vedere dal nuovo Stall Chart, i cicli di attesa dovuti ad accessi in memoria ad alta latenza sono altamente diminuiti (da 25 a 9). Come contro abbiamo aumentato i cicli di barrier (da 19 a 32), in quanto alcuni thread, introducono maggiore latenza per i thread in attesa sulle barriere di sincronizzazione per via della scrittura in memoria globale, relativa al check dei confini.

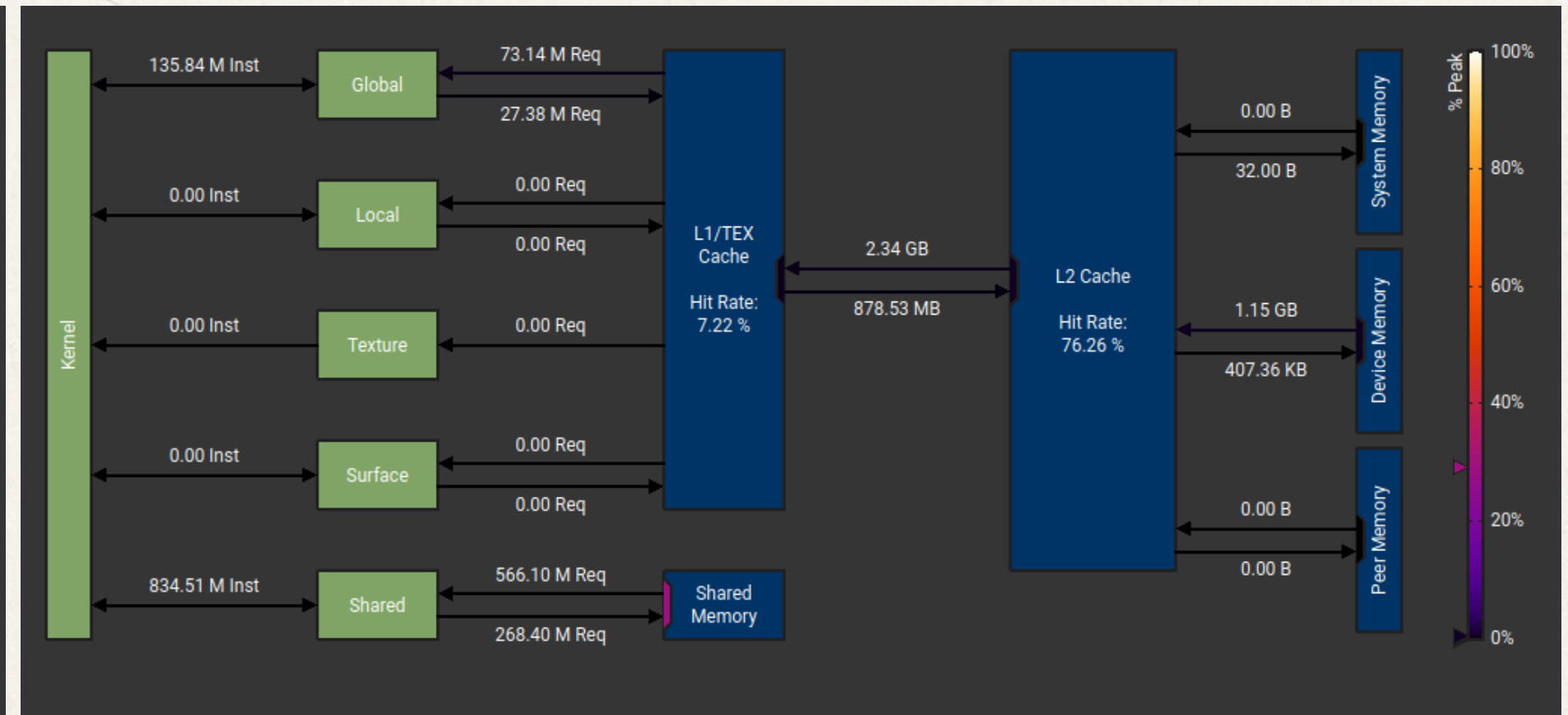


Ottimizzazione Incrementale II

Confronto Accessi in Memoria



Memory Chart V1 - 126K elementi



Memory Chart V2 - 126K elementi

Cambiamento radicale del pattern di accesso ai dati:

- Nel primo Chart il passaggio continuo di dati tra L1 e L2 per ogni confronto ha creato un collo di bottiglia di traffico interno, generando stalli.
- Nel secondo Chart le migliaia di operazioni di scambio avvengono direttamente in shared memory. I thread accedono in DRAM solo in fase di sincronizzazione

NOTA: Disabilitare la cache L1 non determina un aumento delle prestazioni, ma anzi un peggioramento, nonostante il suo poco utilizzo

Pro soluzione adottata:

- Meno transazioni in Memoria Globale
- Speedup del tempo di esecuzione

Contro soluzione adottata:

- Accessi allineati non coalescenti in fase di sincronizzazione (inevitabili ma meno che in versione I)
- Dimensione della SMEM per blocco cresce al crescere della dimensione dell'array, diminuendo l'occupancy e determinando un range ottimale di miglioramento

Confronto Speedup

Esecuzione su GPU NVidia GTX 1660 Super

NOTA:

Esecuzione su GPU NVidia GTX 1660 Super, caratteristiche tecniche:

- Architettura Turing
- 6GB di memoria GDDR6
- 1408 CUDA core
- bus di memoria a 192 bit
- larghezza di banda di 336 GB/s

Cooperative Kernels - Versione Base - Int 32bit

Dim. Array	Tgpu(ms)	Tcpu(ms)	Speedup	Dim. Blocco
2k (8KB)	4.730	8.486	1.794	256
16K (64KB)	35.180	670.935	19.071	256
64K (256KB)	281.895	12749.733	45.229	256
128K (500KB)	1216.623	51289.597	42.157	256
256K (1MB)	6298.442	202178.513	32.100	256
128K (500KB)	1627.017	51289.597	31.466	512
256K (1MB)	8003.337	202178.513	25.563	512

Cooperative Kernels - Shared Memory - Int 32bit

Dim. Array	Tgpu (ms)	Tcpu (ms)	Speedup	Dim. Blocco
2k (8KB)	4.281	8.486	1.982	256
16K (64KB)	30.180	670.935	22.231	256
64K (256KB)	194.136	12749.733	60.472	256
128K (500KB)	691.203	51289.597	72.138	256
256K (1MB)	3630.719	202178.513	53.894	256
128K (500KB)	793.413	51289.597	59.962	512
256K (1MB)	4952.837	202178.513	39.513	512

Analisi:

- Soglia minima per sfruttare efficacemente il parallelismo 2k elementi.
- Numero ottimale di thread per blocco: 256 thread
- Lo speedup rispetto alla versione seriale ha un valore di picco a 500KB per la gestione ottimale della shared memory per blocco.
- Nel caso migliore, l'utilizzo della Shared Memoria porta ad uno speed up di 1,71 rispetto alla versione senza.

Implementazione Algoritmo SIMD

Fase 1: paradigma SIMD, trovo i 4 elementi più piccoli per ogni colonna

A[0] = 13	A[1] = 7	A[2] = 8	A[3] = 11
A[4] =15	A[5] = 6	A[6] = 9	A[7] =10
...	...	...	...

Fase 2: Una volta trovato l'elemento più piccolo per ogni colonna, itero sequenzialmente (SISD) nel registro e trovo il minimo GLOBALE, del quale ho anche l'indice nell'array.

R_MIN[0] =13	R_MIN[1] =6	R_MIN[2] =8	R_MIN[3] =10
IDX_MIN[0]=0	IDX_MIN[1]=5	IDX_MIN[2]=2	IDX_MIN[3]=7

Fase 3: Salvo il valore minimo in un registro apposito, nella posizione $i\%4$, dove i è il numero dell'iterazione. (Da 0 a N).

TEMP[0]= 6	TEMP[1]= ???	TEMP[2]= ???	TEMP[3]= ???
------------	--------------	--------------	--------------

Fase 4: Swappo l'elemento minimo dell'array con l'elemento nella posizione i

A[0] =6	A[1] = 7	A[2] = 8	A[3] = 11
A[4] =15	A[5] = 13	A[6] = 9	A[7] =10

Fase 5: Metto il numero massimo in $A[i]$, per fare in modo che l'elemento minimo non venga considerato

A[0] =MAX	A[1] = 7	A[2] = 8	A[3] = 11
A[4] =15	A[5] = 13	A[6] = 9	A[7] =10

Ogni volta che $i \% 4 = 3$, allora significa che ho il registro TEMP pieno

TEMP[0]= 6	TEMP[1]= 7	TEMP[2]= 8	TEMP[3]= 9
------------	------------	------------	------------

E il mio array avrà questa forma

A[0] =MAX	A[1] = MAX	A[2] =MAX	A[3] = MAX
A[4] =15	A[5] = 13	A[6] = 11	A[7] =10

Fase 5: Di conseguenza, ogni volta che $i\%4=3$, posso fare la store del contenuto di TEMP nel mio array A, a partire dal corretto indice [che sarebbe $\text{floor}(i/4)$]

A[0] =6	A[1] = 7	A[2] = 8	A[3] =9
A[4] =15	A[5] = 13	A[6] = 11	A[7] =10

Da qui si vede che ogni 4 iterazioni possiamo ingorare un blocco in più.
Blocchi da ignorare = $\text{floor}(i/4)$

Analisi delle prestazioni

12th Gen Intel(R) Core(TM) i7-1255U

Istruzioni SIMD: SSE2

Elementi: interi a 32 bit (sinistra), char da 8 bit

Vediamo che le ottimizzazioni del compilatore permettono di arrivare ad uno speed up fino a 15. Il compilatore ha più “margine di miglioramento” nel codice SIMD rispetto al codice sequenziale

N el da 4 byte	-00	-01	-02	-03
64K	x2	x7.74	x15.5	x11
128K	x2.36	x7.13	x13.7	x15
256K	x2.15	x8.65	x13.7	x13.4

N el da 1 byte	-00	-01	-02	-03
64K	X2,83	X8,12	X8,69	X10,21
128K	X2,52	X9,17	x10.53	X10,68
256K	X3,28	x9.81	x11.52	X11

IPC di 3.82 indica molte istruzioni in esecuzione per ciclo. Notiamo alto backend cycle idle. Questo indica stalli dovuti a dipendenza di dato o di risorse. Questo suggerisce dipendenze tra istruzioni, come ci aspettavamo vista la natura del problema in analisi.

249,57 msec	task-clock	#	0,995 CPUs utilized
0	context-switches	#	0,000 /sec
0	cpu-migrations	#	0,000 /sec
94	page-faults	#	376,642 /sec
3.494.984.318	instructions	#	3,82 insn per cycle
		#	0,15 stalled cycles per insn (83,18%)
915.361.497	cycles	#	3,668 GHz (83,17%)
1.398.237	stalled-cycles-frontend	#	0,15% frontend cycles idle (83,30%)
536.732.067	stalled-cycles-backend	#	58,64% backend cycles idle (83,56%)
135.846.874	branches	#	544,315 M/sec (83,58%)
170.951	branch-misses	#	0,13% of all branches (83,22%)
0,250760183 seconds time elapsed			
0,248841000 seconds user			
0,001998000 seconds sys			

Conclusioni e sviluppi futuri

CUDA

L'utilizzo dei cooperative groups ha portato ottimi risultati ma l'utilizzo di questo API può compromettere la scalabilità del codice con un numero elevato di elementi, fornendo possibilità limitate nel miglioramento di occupancy e latency hiding.

Un'alternativa per ovviare questo problema potrebbe essere usare i CUDA Stream.

Inoltre abbiamo anche visto che il codice può essere modificato per evitare il conflict nella shared memory, il che potrebbe aumentare ulteriormente le prestazioni.

SIMD

Vedendo l'analisi precedentemente fatta con perf, si può ipotizzare che utilizzando registri estesi più grandi (e.g. AVX512) si potrebbero ottenere migliori prestazioni. L'analisi fatta con perf mostra anche come la forte dipendenza di dato tra le istruzioni peggiori le performance. Elaborando ancora più dati in un singolo istante (usando quindi più registri) si potrebbe ovviare questo problema.

CONCLUSIONI

Nonostante il problema in esame non sia esattamente "embarassingly parallel", questo progetto ha mostrato come la corretta decomposizione del problema e l'utilizzo di programmi che sfruttano opportunamente le risorse hardware disponibili permettono di ottenere prestazioni maggiorate. In generale abbiamo osservato che i programmi con un'elevata necessità di sincronizzazione che si estende oltre i limiti del blocco, possono risultare critici e richiedono adattamenti per poter essere eseguiti correttamente secondo il paradigma SIMT.